



CS-2001
Data Structures

Spring 2023
Abstract Data Types and Data Structures

Rizwan Ul Haq
Assistant Professor
FAST-NU
rizwan.haq@nu.edu.pk

Abstract Data Types

Lecture 03

Todays Lecture

- Data
- Data Abstraction and Abstract Data Type
- Data Structure
- Abstraction Levels
- Abstract Data Types
- Data Structure Vs ADT
- List
- Array Based Implementation of List

What do we mean by Data?

- The objects that are manipulated
- The information that is processed by computer program

Abstraction

Data Abstraction & Abstract Data Types (ADTs)

Data Abstraction

- A **separation between computer** and **ours' view** of data
- Goal is to hide complexity
- **Example:** an integer
- More formally we can say that:

  The separation of a data type's logical properties from its implementation is known as **data abstraction**

Some basic questions

- What is an **integer**?
  - What are the operations that can be performed on Integers?
- What is a **character**?
  - What are the operations that can be performed on characters?
- How an **integer is represented in memory?**
  - How the integer based operations are performed?
- How a **rational number is represented?**
  - How different operations are performed?

Abstract Data Type (ADT)

- **ADT:** A **definition** for a data type solely in terms of a **set of values** and a **set of operations** on that data type (independently of any implementation details)
- Each ADT operation is defined by its inputs and outputs
- **Example: Flight reservation system**
- **Basic operations:** find an empty seat, reserve a seat, cancel a seat assignment
- Why **"abstract?"**
  - Data, operations, and relations are studied independent of implementation
  - What, not how is the focus

Why Abstract?

- Specify the **operations** of the data structure and leave **implementation** details for later
- many, many different Data Structures
  - picking the right one for the job is an important step in design
- High level languages often provide built in ADTs,
  - The C++ STL, the Java standard library

Abstract Data Types

- In Object Oriented Programming **data** and the **operations** that manipulate that data are grouped together in classes
- Abstract Data Types (ADTs) allow various operations on the data to access and change it

Abstraction levels

- **Application Level or user Level**
  - A way of modeling real-life data in a specific context also called the problem domain. We Use ADT
- **Logical Level**
  - An abstract view of the data values and the set of operations to manipulate them. We define ADT
- **Implementation Level**
  - A specific representation of the structure to hold the data items and the coding of the operations in a programming language

Abstraction in Programming Language

- While programming in a high-level language we are not that bothered about low level representation

Abstract Data Types

- Abstract data-types allow us to take our abstraction level, one-step further
- Instead of focusing on how a particular data-structure can be implemented, focus is on what should be chosen to solve the problem
- **ENCAPSULATION:** Hide Implementation details

Data Structure

- A **data structure** is the physical implementation of an ADT
  - Each operation associated with the ADT is implemented by one or more subroutines in the implementation

- **Data structure** usually refers to an organization of data in main memory

Selecting a Data Structure

- **Analyze** the problem to determine the resource constraints a solution must meet
- **Determine** the basic operations that must be supported. Quantify the resource constraints for each operation.
- **Select** the data structure that best meets these requirements

Data Structure Philosophy

- Each data structure has costs and benefits.
- Rarely is one data structure better than another in all situations
- A data structure requires:
  - **space** for each data item it stores,
  - **time** to perform each basic operation,
  - programming **effort**

Common Operations on Data Structure

- **Traversing**
  - Accessing each record exactly once so that certain items in the record may be processed
- **Searching**
  - Finding the location of the record with the given key value or finding the location of all records which satisfy one or more conditions
- **Insertion**
  - Adding a new record to the structure

Common Operations on Data Structure

- **Deletion**
  - Removing a record from the structure
- **Sorting**
  - Arrange the records in a logical order
- **Merging**
  - Combining records from two or more files or data structures into one

Abstract Data Type Vs Data Structure

- An **Abstract data** type is **implementation independent**
- A **Data Structure** is **implementation dependent**
- A Data structure is how we **implement** the data in an abstract data type whose values have component parts
- The **operation** on an abstract data type are translated into **algorithms** on the data structure

Implementation of ADT

- An implementation of ADT consists of **storage structures** to store the data items and **algorithms** for basic operation

Data Structures, Abstract Data Types, and Implementations

- Consider example of an airplane flight with 10 seats to be assigned



- Tasks
  - List available seats
  - Reserve a seat
- How to store, access data?
  - 10 individual variables

Use 10 individual variables

Algorithm to List available seats

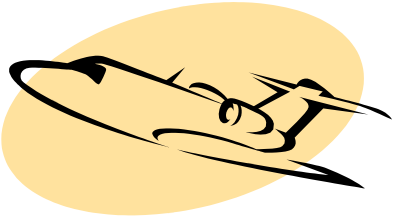
```
1. If seat1 == ‘ ’:
        display 1
2. If seat2 == ‘ ’:
        display 2
.
.
.
10. If seat10 == ‘ ’:
        display 10
```

Algorithm to Reserve a seat

```
1. Set DONE to false
2. If seat1 ==‘ ’:
        print “do you want seat #1??”
        Get answer
        if answer==‘Y’:
                set seat1 to ‘X’
                set DONE to True
3. If seat2 ==‘ ’:
        print “do you want seat #2??”
        Get answer
        if answer==‘Y’:
                set seat2 to ‘X’
                set DONE to True
…
```

Data Structures, Abstract Data Types, and Implementations

- Consider example of an airplane flight with 10 seats to be assigned



- Tasks
  - List available seats
  - Reserve a seat
- How to store, access data?
  - An array of variables

Use Array

- **Algorithm to List available seats**

For number ranging from 0 to max_seats-1, do:

    If seat[number] == ‘ ’:

        Display number

- **Algorithm to Reserve a seat**

Read in number of seat to be reserved

If seat[number] is equal to ‘ ’:

    set seat[number] to ‘X’

Else

    Display a message that the seat having this number is occupied

Seat Reservation ADT

- This simple example does illustrate the concept of an **Abstract Data Type**

- ADT consists of
  - The collection of data items
  - Basic operation that must be performed on them

- In the example, a collection of data is a list of seats
- The basic operations are
  - Scan the list to determine which seats are occupied
  - Change seat's status

List ADT & Implementation

Implementation via arrays

Consider Every Day Lists

- Groceries to be purchased
- Job to-do list
- List of assignments for a course

- Can you name some others??

Properties of Lists

- Can have a single element
- Can have **no** elements
- There can be lists of lists

- We will look at the list as an abstract data type
  - Homogeneous
  - Finite length
  - Sequential elements

Basic Operations

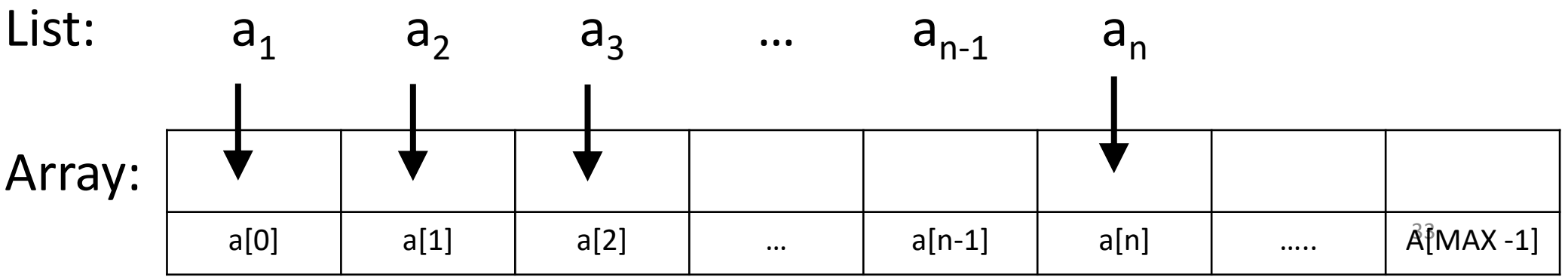
- **Construct** an empty list
- Determine whether or not **empty**
- **Insert** an element into the list
- **Delete** an element from the list
- **Traverse** (iterate through) the list to
  - Modify
  - Output
  - Search for a specific value
  - Copy or save
  - Rearrange

Designing a List Class

- Should contain at least the following function members
  - Constructor
  - isEmpty()
  - insert()
  - delete()
  - display()
- Implementation involves
  - Defining data members
  - Defining function members from design phase

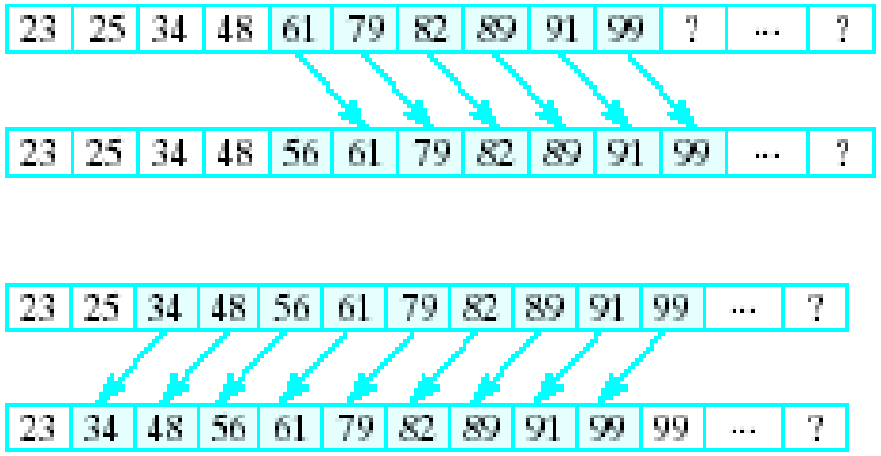
Array-Based Implementation of Lists

- An array is a feasible choice for storing list elements
  - Element are sequential
  - It is a commonly available data type
  - Algorithm development is easy
- Normally sequential orderings of list elements match with array elements



Implementing Operations

- **Size:** no of elements stored currently in an array
- **Capacity:** total elements that array can store
- **Constructor**
  - Static array allocated at compile time, size=0
- **isEmpty**
  - Check if size == 0
- **Traverse**
  - Use a loop from 0th element to size – 1
- **Insert**
  - Shift elements to right of insertion point
- **Delete**
  - Shift elements back



Also adjust **size**

Insertion in static array

Algorithm for Insert

//insert ***item*** at position ***pos*** in a list
//First check if there is room in the array
1. If ***size*** is equal to ***capacity***
    issue an error message and terminate this operation
//Next check if the position is legal
2. If ***pos*** < 0 or ***pos*** > ***size***
    Signal an illegal insert position and terminate this operation
  Otherwise:
    //Shift array elements right to make room for ***item***
    a. For ***i*** ranging from ***size - 1*** down to ***pos + 1***
        array[*i*] *= array[i-1*]
    //Now insert ***item*** at position ***pos*** and increase list size
    b. array[pos] = item
    c. ***size++***

Analysis

- The efficiency of this insert algorithm obviously depends on the number of array elements that must be shifted to make room for the new item-that is, on the number of times that the instructions in the loop are executed
- In the worst case, the new item must be inserted at the beginning of the list, which requires shifting all of the array elements
- When items may be inserted at any positions in the list, on the average, one half of the array elements must be shifted to make room for a new item.
- Thus, we can say that, in both the worst case and on average, the computing times for this insert algorithm grows linearly with the size of the list. Using the big-O notation introduced we can express the computing time as **O(n)**, where **n** is the **list size**
- The best case occurs when the new item is inserted at the end of the list
- Insertions can be carried out in constant time; therefore, the best-case computing time is **0(1)**

Deletion in static array

Algorithm for Delete

//Delete the element at position ***pos*** in a list
//First check that list isn't empty
1. If ***size*** is ***0***
        issue an error message and terminate this operation
//Next check that ***pos*** is legal
2. If ***pos*** < 0 or ***pos*** ≥ ***size***
        Issue and error message and terminate this operation
   Otherwise:
        //Shift array elements left to fill the gap
        a. For ***i*** ranging from ***pos*** to ***size - 2***
                array[*i*] = *array[i+1*]
        //Decrease the list size
        b. ***size--***

Analysis

- The computing time of such a function is easily seen to be the same as that of an insert function-O(n) in the worst and average cases and ***0(1)*** in the best case

List Class with Static Array - Problems

- Stuck with "one size fits all"
  - Could be wasting space
  - Could run out of space
- Better to have instantiation of specific list specify what the capacity should be
- Thus we consider creating a **List** class with dynamically-allocated array

Dynamic-Allocation for List Class

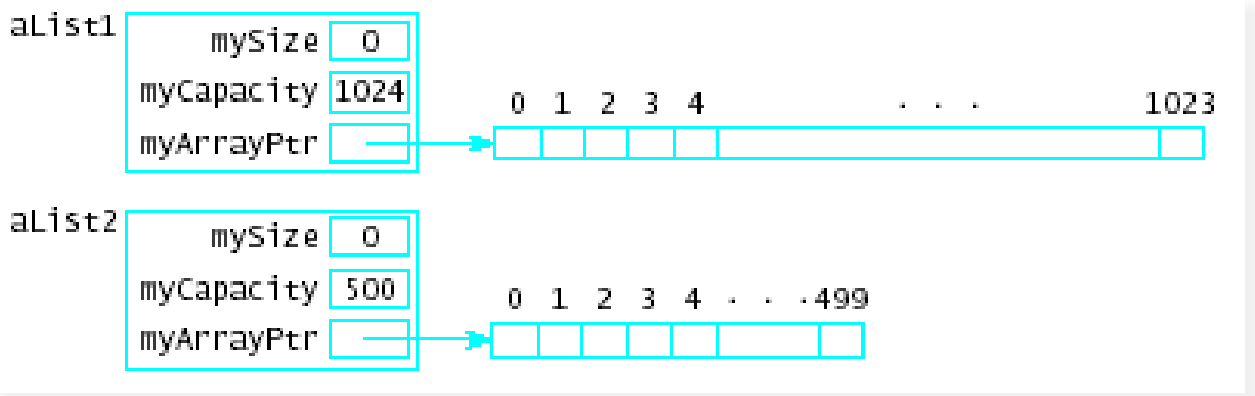
- Changes required in data members
  - Eliminate const declaration for MAX
  - Add variable data member to store capacity specified by client program

- Little or no changes required for
  - **isEmpty()**
  - **display()**
  - **delete()**
  - **insert()**

Dynamic-Allocation for List Class

- Now possible to specify different sized lists

```
cin >> maxListSize;
List aList1 (maxListSize);
List aList2 (500);
```




```
List * aList1 ;
aList1 = new List [5];
```

Improvements to our List Class

- Problem 1: Array used has fixed capacity
  Solution:
  - If larger array needed during program execution
  - Allocate, copy smaller array to the new one

- Problem 2: Class bound to one type at a time
  Solution:
  - Create multiple List classes with differing names
  - Use class template

Inefficiency of Array-Implemented List

- **Insert(), erase()** functions inefficient for dynamic lists
  - Those that change frequently
  - Those with many insertions and deletions

So …

We will look for an alternative implementation.